

```
# CODIGO_CRU_TRIAGEM_DELTA
```

```
## PILAR 1 - Escopo do Loop de Processamento do `new_buffering` (`worker.js`)
```

```
### `__deltaEnqueueNewLeadTimerToDiskSync` + `__deltaRunNewLeadsTimerPump`
```

```
``txt
```

```
L15052: function __deltaEnqueueNewLeadTimerToDiskSync({ nome, thread_key, dueAt, delayMs, forceEnqueue = false, queueLane }) {
L15053:   try {
L15054:     const n = String(nome || "").trim();
L15055:     const tk = String(thread_key || "").trim();
L15056:     if (!n || !tk) return false;
L15057:     if (!forceEnqueue && !__deltaCanEnqueueNewLeadTimerFromDiskSync(n, tk)) return false;
L15058:     if (__deltaHasPendingNewLeadTimerOnDiskSync(n, tk)) return false;
L15059:     __deltaEnsureNewLeadsQueueDirsSync(n);
L15060:     const rec = {
L15061:       ts: Date.now(),
L15062:       type: 'new_lead_timer',
L15063:       nome: n,
L15064:       thread_key: tk,
L15065:       delayMs: Math.max(0, Number(delayMs || 0) || 0),
L15066:       dueAt: Math.max(0, Number(dueAt || 0) || 0),
L15067:       queue_lane: String(queueLane || 'primary').trim() || 'primary',
L15068:       retry_reason: String(retryReason || "").trim() || null,
L15069:     };
L15070:     fs.appendFileSync(__deltaNewLeadsOutboxPath(n), JSON.stringify(rec) + '\n', 'utf8');
L15071:     return true;
L15072:   } catch {
L15073:     return false;
L15074:   }
L15075: }

L15123: async function __deltaRunNewLeadsTimerPump(nome) {
L15124:   const n = String(nome || "").trim();
L15125:   if (!n) return;
L15126:   if (__deltaNewLeadsTimerInFlight.has(n)) return; // 1 timer por vez POR CONTA
L15127:   try {
L15128:     const outbox = __deltaNewLeadsOutboxPath(n);
L15129:     if (!fs.existsSync(outbox)) return;
L15130:     const cursor = __deltaReadNewLeadsCursorSync(n);
L15131:     const offset = Math.max(0, Number(cursor && cursor.offset || 0) || 0);
L15132:
L15133:     let fd = null;
L15134:     try {
L15135:       fd = fs.openSync(outbox, 'r');
L15136:       const st = fs.fstatSync(fd);
L15137:       const size = Number(st && st.size || 0) || 0;
L15138:       if (offset >= size) return;
L15139:
L15140:       const maxChunk = 64 * 1024;
L15141:       const toRead = Math.min(maxChunk, size - offset);
L15142:       const buf = Buffer.allocUnsafe(toRead);
L15143:       const bytes = fs.readSync(fd, buf, 0, toRead, offset);
```

```

L15144: const txt = buf.slice(0, bytes).toString('utf8');
L15145: const nl = txt.indexOf('\n');
L15146: if (nl === -1) return; // linha incompleta
L15147: const line = txt.slice(0, nl).trim();
L15148: const nextOffset = offset + Buffer.byteLength(txt.slice(0, nl + 1), 'utf8');
L15149: if (!line) {
L15150:   try { __deltaScrubNewLeadTimersOutboxSync(n, { consumeUntilOffset: nextOffset }); } catch { __deltaWriteNewLeadsCursorSync(n, nextOffset); }
L15151:   __deltaKickNewLeadsTimerPump(n);
L15152:   return;
L15153: }
L15154: let rec = null;
L15155: try { rec = JSON.parse(line); } catch { rec = null; }
L15156: if (!rec || String(rec.type || "").trim() !== 'new_lead_timer') {
L15157:   try { __deltaScrubNewLeadTimersOutboxSync(n, { consumeUntilOffset: nextOffset }); } catch { __deltaWriteNewLeadsCursorSync(n, nextOffset); }
L15158:   __deltaKickNewLeadsTimerPump(n);
L15159:   return;
L15160: }
L15161: const tk = String(rec.thread_key || "").trim();
L15162: if (!tk) {
L15163:   try { __deltaScrubNewLeadTimersOutboxSync(n, { consumeUntilOffset: nextOffset }); } catch { __deltaWriteNewLeadsCursorSync(n, nextOffset); }
L15164:   __deltaKickNewLeadsTimerPump(n);
L15165:   return;
L15166: }
L15167:
L15168: // Se o thread já ficou ativo (ou em voo), não faz sentido segurar represa: consome e segue.
L15169: const stThread = __deltaGetThreadState(n, tk);
L15170: if (stThread && (stThread.status === 'active' || stThread.status === 'hands_in_progress' || stThread.inFlight)) {
L15171:   try { __deltaScrubNewLeadTimersOutboxSync(n, { consumeUntilOffset: nextOffset }); } catch { __deltaWriteNewLeadsCursorSync(n, nextOffset); }
L15172:   __deltaKickNewLeadsTimerPump(n);
L15173:   return;
L15174: }
L15175:
L15176: const dueAt = Math.max(0, Number(rec.dueAt || 0) || 0);
L15177: const remainMs = Math.max(0, dueAt ? (dueAt - Date.now()) : (Math.max(0, Number(rec.delayMs || 0) || 0)));
L15178: try {
L15179:   console.log('[FORENSIC_BUFFER] ' + JSON.stringify({
L15180:     event: 'new_lead_reservoir_timer_begin',
L15181:     scheduled_at: Date.now(),
L15182:     account_login: n,
L15183:     thread_key: tk,
L15184:     remain_ms: remainMs,
L15185:     dueAt: dueAt || null,
L15186:   }));
L15187: } catch {}
L15188:
L15189: const handle = setTimeout(() => {
L15190:   __deltaNewLeadsTimerInFlight.delete(n);
L15191:   // Consome da represa PRIMEIRO (para liberar próximo timer).
L15192:   let scrubStats = null;
L15193:   try { scrubStats = __deltaScrubNewLeadTimersOutboxSync(n, { consumeUntilOffset: nextOffset, dropThreadKey: true }); } catch {}
L15194:   if (!scrubStats || scrubStats.ok !== true) {
L15195:     try { __deltaWriteNewLeadsCursorSync(n, nextOffset); } catch {}

```

```

L15196:     }
L15197:   try {
L15198:     console.log('[FORENSIC_BUFFER] ' + JSON.stringify({
L15199:       event: 'new_lead_reservoir_timer_fired',
L15200:       fired_at: Date.now(),
L15201:       account_login: n,
L15202:       thread_key: tk,
L15203:       dueAt: dueAt || null,
L15204:       scrub_removed_total: Number(scrubStats && scrubStats.removed_total || 0) || 0,
L15205:       scrub_removed_thread: Number(scrubStats && scrubStats.removed_thread || 0) || 0,
L15206:       scrub_kept: Number(scrubStats && scrubStats.kept || 0) || 0,
L15207:     }));
L15208:   } catch {}
L15209:   // Downstream: dispara a ação no pipeline soberano (ctrl.virtus) em background.
L15210:   try {
L15211:     Promise.resolve()
L15212:       .then(() => __deltaHandleBufferedThreadTimer(n, tk, { reason: 'initial' }))
L15213:       .catch(() => {});
L15214:   } catch {}
L15215:   // Ativa o próximo lead novo da represa imediatamente.
L15216:   try { __deltaKickNewLeadsTimerPump(n); } catch {}
L15217: }, Math.max(0, remainMs));
L15218: handle.unref?.();
L15219: __deltaNewLeadsTimerInFlight.set(n, handle);
L15220: } finally {
L15221:   try { if (fd) fs.closeSync(fd); } catch {}
L15222: }
L15223: } catch {}
L15224:}
```

```

### `ingestLeadEvents` (`new\_buffering` capture -> payload -> represa)

```

```txt
L18404:   const st = __deltaGetOrCreateThreadState(nome, threadKey);
L18405:   if (__deltaIsKnownProcessedStatus(st.status)) {
L18406:     try {
L18407:       __forensicEdgeEmit({
L18408:         account_login: String(nome || ''),
L18409:         thread_key: threadKey,
L18410:         flow_stage: 'discard_filter_triggered',
L18411:         details: {
L18412:           reason: 'legacy_historical',
L18413:           state_status: String(st.status || ''),
L18414:           op: op || null,
L18415:           text_preview: String(texto || '').slice(0, 220)
L18416:         }
L18417:       });
L18418:     } catch {}
L18419:     __deltaAppendPendingJsonlSync({
L18420:       event: 'lead_skip_forense_estado_conhecido',
L18421:       server_id: serverId || null,

```

```

L18422:     account_login: String(nome || ""),
L18423:     thread_key: threadKey,
L18424:     texto_limpo: texto,
L18425:     cidade: st.city || null,
L18426:     operacao_meta: op || 'message',
L18427:     mensagem_seq: Number(st.seq || 0) || 0,
L18428:     dispatch_ct: false,
L18429:     queue_mode: 'capture_only',
L18430:     flow_stage: 'skip_known_processed_state',
L18431:     state_status: String(st.status || ""),
L18432:     message_at: nowMs,
L18433:     ...networkCtx
L18434: });
L18435: try { __deltaThreadStateMap.delete(__deltaThreadStateKey(nome, threadKey)); } catch {}
L18436: continue;
L18437: }
L18438: if (!dedupMetaId && __deltalsRecentDuplicate(st, texto, op, nowMs)) {
L18439:   try {
L18440:     __forensicEdgeEmit({
L18441:       account_login: String(nome || ""),
L18442:       thread_key: threadKey,
L18443:       flow_stage: 'discard_filter_triggered',
L18444:       details: {
L18445:         reason: 'sha1_text_collision',
L18446:         op: op || null,
L18447:         window_ms: Number(DELTA_RECENT_DEDUP_WINDOW_MS || 0) || 0,
L18448:         text_preview: String(texto || "").slice(0, 220)
L18449:       }
L18450:     });
L18451:   } catch {}
L18452:   continue;
L18453: }
L18454: if (__deltalsRecentInsertUpsertMirrorDuplicate(st, texto, op, nowMs)) {
L18455:   try {
L18456:     __forensicEdgeEmit({
L18457:       account_login: String(nome || ""),
L18458:       thread_key: threadKey,
L18459:       flow_stage: 'discard_filter_triggered',
L18460:       details: {
L18461:         reason: 'insert_upsert_mirror_duplicate',
L18462:         op: op || null,
L18463:         text_preview: String(texto || "").slice(0, 220)
L18464:       }
L18465:     });
L18466:   } catch {}
L18467:   continue;
L18468: }
L18469: const msg = __deltaPushMessageToState(st, { text: texto, op, at: nowMs });
L18470: const msgSeq = Number(msg && msg.seq || st.seq || 0) || 0;
L18471:
L18472: __deltaAppendPendingJsonlSync({
L18473:   event: 'lead_capturado_buffer',

```

```

L18474:     server_id: serverId || null,
L18475:     account_login: String(nome || ""),
L18476:     thread_key: threadKey,
L18477:     texto_limpo: texto,
L18478:     dedup_meta_id: dedupMetaId || null,
L18479:     meta_message_id: metaIds && metaIds.msgId ? String(metaIds.msgId) : null,
L18480:     meta_offline_threading_id: metaIds && metaIds.offlineId ? String(metaIds.offlineId) : null,
L18481:     cidade: st.city || null,
L18482:     operacao_meta: op,
L18483:     mensagem_seq: msgSeq,
L18484:     dispatch_ct: false,
L18485:     queue_mode: 'capture_only',
L18486:     flow_stage: String(st.status || 'new_buffering'),
L18487:     message_at: nowMs,
L18488:     ...networkCtx
L18489: });
L18490:
L18542: // Arquitetura 2 camadas (Gemini): novo lead NÃO arma timer em paralelo por thread.
L18543: // Enfileira na REPRESA (1 timer por vez por conta) e só depois dispara para ctrl.virtus.
L18544: if (st.status !== 'hands_in_progress' && !st.timerHandle && !(Number(st.timerDueAt || 0) > 0)) {
L18545:     const delayMs = __deltaRandInt(DELTA_NEW_CHAT_TIMER_MIN_MS, DELTA_NEW_CHAT_TIMER_MAX_MS);
L18546:     st.timerReason = 'initial';
L18547:     st.timerDueAt = Date.now() + delayMs;
L18548:     st.updatedAt = nowMs;
L18549:     __deltaSchedulePersistThreadState();
L18550:     __deltaEnqueueNewLeadTimerToDiskSync({ nome: String(nome || ""), thread_key: threadKey, delayMs, dueAt: st.timerDueAt });
L18551:     __deltaKickNewLeadsTimerPump(String(nome || ""));
L18552:     try {
L18553:         logger.info('[DELTA][BUFFER] novo chat detectado; timer armado', {
L18554:             nome: String(nome || ""),
L18555:             thread_key: threadKey,
L18556:             delayMs,
L18557:             dueAt: st.timerDueAt,
L18558:             transport: String(transport || "")
L18559:         });
L18560:     } catch {}
L18561: } else {
L18562:     st.updatedAt = nowMs;
L18563:     __deltaSchedulePersistThreadState();
L18564: }

```

PILAR 2 - Método de Ativação Física e Abertura do Hands (`virtusDelta.js`)

`runWrongThreadGuard` + `openThreadByClick`

```txt

```

L2197: async function runWrongThreadGuard(page, threadKey, { forensicAccountLogin = null, stage = "post_click", requireCom
L2198: const t = String(threadKey || "").trim();
L2199: const expectedTarget = `/messages/t/${t}`;
L2200: const currentUrl = String(page && page.url ? page.url() : "").trim();
L2201: const urlMatches = !(currentUrl && currentUrl.includes(expectedTarget));

```

```

L2202:
L2203: let composerCheck = { ok: true, composer_count: null, active_sidebar_href: null };
L2204: if (requireComposer) {
L2205: composerCheck = await page.evaluate((threadId) => {
L2206: const isVisible = (el) => {
L2207: if (!el) return false;
L2208: const r = el.getBoundingClientRect();
L2209: return r && r.width > 1 && r.height > 1;
L2210: };
L2211:
L2212: const composers = Array.from(document.querySelectorAll('div[data-lexical-editor="true"]')).filter(isVisible);
L2213: const active = document.querySelector('a[aria-current="page"][href], [aria-current="page"] a[href]');
L2214: const activeHref = String((active && active.getAttribute("href")) || "").trim();
L2215: const expectedRe = new RegExp(`/messages/(?e2ee)?t/${String(threadId).replace(/[/.*+?^${}()|[\]\]/g, "\\$&")}(?:/|$)`);
L2216: const sidebarMatchesThread = !(activeHref && expectedRe.test(activeHref));
L2217: return {
L2218: ok: composers.length === 1 && sidebarMatchesThread,
L2219: composer_count: composers.length,
L2220: active_sidebar_href: activeHref || null
L2221: };
L2222: }, t).catch(() => ({ ok: false, composer_count: null, active_sidebar_href: null }));
L2223: }
L2224:
L2225: if (urlMatches && composerCheck && composerCheck.ok) {
L2226: return {
L2227: ok: true,
L2228: current_url: currentUrl,
L2229: expected_target: expectedTarget,
L2230: composer_count: composerCheck.composer_count
L2231: };
L2232: }
L2233:
L2234: const reason = !urlMatches
L2235: ? "URL_mismatch_preventing_cross_routing"
L2236: : "composer_signature_mismatch";
L2237: try {
L2238: console.log(JSON.stringify({
L2239: timestamp: Date.now(),
L2240: flow_stage: "wrong_thread_guard_blocked",
L2241: thread_key: t,
L2242: current_url: currentUrl,
L2243: expected_target: expectedTarget,
L2244: stage,
L2245: reason,
L2246: composer_count: composerCheck && composerCheck.composer_count,
L2247: active_sidebar_href: composerCheck && composerCheck.active_sidebar_href
L2248: }));
L2249: } catch (__) {}
L2250: try {
L2251: __forensicEdgeEmit({
L2252: account_login: forensicAccountLogin,
L2253: thread_key: t,

```

```

L2254: flow_stage: "wrong_thread_guard_blocked",
L2255: details: {
L2256: tag: "FORENSIC_DOM_REVERSE",
L2257: stage,
L2258: reason,
L2259: current_url: currentUrl || null,
L2260: expected_target: expectedTarget,
L2261: composer_count: Number(composerCheck && composerCheck.composer_count || 0) || 0,
L2262: active_sidebar_href: composerCheck && composerCheck.active_sidebar_href ? String(composerCheck.active_sidebar_href) : null,
L2263: ts_ms: Date.now()
L2264: }
L2265: });
L2266: } catch (_) {}
L2267: return {
L2268: ok: false,
L2269: error: "wrong_thread_guard_blocked",
L2270: reason,
L2271: current_url: currentUrl,
L2272: expected_target: expectedTarget
L2273: };
L2274:}
L2275:async function openThreadByClick(page, threadKey, { maxScrollSteps = 16, forensicAccountLogin = null } = {}) {
L2276: const t = String(threadKey || "").trim();
L2277: if (!t) throw new Error("thread_key_empty");
L2278: const isStable = await waitForMessagesBootStable(page, "agent_outbox_hydration_check").catch(() => false);
L2279: if (!isStable) {
L2280: try {
L2281: __forensicEdgeEmit({
L2282: account_login: forensicAccountLogin,
L2283: thread_key: t,
L2284: flow_stage: "messages_boot_not_stable",
L2285: details: {
L2286: tag: "FORENSIC_DOM_REVERSE",
L2287: reason: "agent_outbox_hydration_check_failed",
L2288: ts_ms: Date.now(),
L2289: }
L2290: });
L2291: } catch (_) {}
L2292: }
L2293: return { ok: false, error: "messages_boot_not_stable" };
L2294: }
L2295: }
L2296: }
L2297: try {
L2298: await page.waitForFunction(
L2299: () => {
L2300: const hrefs = Array.from(document.querySelectorAll('a[href*="/messages"]'))
L2301: .map((a) => String(a.getAttribute("href") || ""))
L2302: .filter(Boolean);
L2303: const nonNew = hrefs.filter((h) => !h.includes("/messages/new"));
L2304: return nonNew.length >= 1;
L2305: },
L2306: { timeout: 8000 }
)
}

```

```

L2307:);
L2308: } catch (_) {}
L2309:
L2310: const primaryCardSelector = `div[role="row"] a[href*="/messages/t/${t}"]`;
L2311: const cardSelectors = [
L2312: primaryCardSelector,
L2313: `div[role="row"] a[href="/messages/t/${t}/"]`,
L2314: `div[role="row"] a[href="/messages/t/${t}"]`,
L2315: `div[role="row"] a[href*="/messages/e2ee/t/${t}"]`,
L2316:];
L2317:
L2318: for (let i = 0; i < maxScrollSteps; i++) {
L2319: for (const cardSelector of cardSelectors) {
L2320: const cardElement = cardSelector === primaryCardSelector
L2321: ? await page.waitForSelector(cardSelector, { timeout: i === 0 ? 5000 : 1200 }).catch(() => null)
L2322: : await page.$(cardSelector).catch(() => null);
L2323: if (cardElement) {
L2324: try {
L2325: const isCurrentPage = await cardElement
L2326: .evaluate((el) => {
L2327: if (!el) return false;
L2328: if (el.getAttribute("aria-current") === "page") return true;
L2329: return Boolean(el.closest("[aria-current="page"]));
L2330: })
L2331: .catch(() => false);
L2332: if (isCurrentPage) {
L2333: const guard = await runWrongThreadGuard(page, t, {
L2334: forensicAccountLogin,
L2335: stage: "already_open_prevent_cross_routing",
L2336: requireComposer: true
L2337: });
L2338: if (!guard.ok) return guard;
L2339: return { ok: true, scrolled: i, matched_selector: cardSelector, already_open: true, skipped_click: true, hydrated: true };
L2340: }
L2341: } catch (_) {}
L2342:
L2343: await humanPause("preThreadClick", "pre_thread_card_click");
L2344: try { await cardElement.evaluate((el) => { try { el.scrollIntoView({ block: 'center', inline: 'nearest', behavior: 'instant' }) });
L2345: let openedByNavigation = false;
L2346: let clickPlan = null;
L2347: try { clickPlan = await computeVisibleThreadCardClickPlan(cardElement); } catch (_) {}
L2348: const points = (clickPlan && clickPlan.ok && Array.isArray(clickPlan.points)) ? clickPlan.points : [];
L2349: for (const p of points.slice(0, 2)) {
L2350: const px = Number(p && p.x);
L2351: const py = Number(p && p.y);
L2352: if (!Number.isFinite(px) || !Number.isFinite(py)) continue;
L2353: try {
L2354: await page.mouse.move(px, py, { steps: 6 });
L2355: await page.mouse.click(px, py, { delay: 100 });
L2356: } catch (_) { continue; }
L2357: }
L2358: try {

```



```

L2359: (threadId) => {
L2360: const path = String(location.pathname || "");
L2361: return path.includes("/messages") && path.includes(`/t/${threadId}`);
L2362: },
L2363: { timeout: 1800 },
L2364: t
L2365:);
L2366: openedByNavigation = true;
L2367: break;
L2368: } catch (_) {}
L2369: }
L2370: if (!openedByNavigation) {
L2371: try {
L2372: await cardElement.focus().catch(() => {});
L2373: await page.keyboard.press("Enter").catch(() => {});
L2374: await page.waitForFunction(
L2375: (threadId) => {
L2376: const path = String(location.pathname || "");
L2377: return path.includes("/messages") && path.includes(`/t/${threadId}`);
L2378: },
L2379: { timeout: 1500 },
L2380: t
L2381:).catch(() => {});
L2382: openedByNavigation = await page.evaluate((threadId) => {
L2383: const p = String(location.pathname || "");
L2384: return p.includes("/messages") && p.includes(`/t/${threadId}`);
L2385: }, t).catch(() => false);
L2386: } catch (_) {}
L2387: }
L2388: await humanPause("postThreadOpen", "post_thread_card_click");
L2389: try {
L2390: await page.waitForSelector('div[data-lexical-editor="true"]', { timeout: 5000 });
L2391: } catch (_) {}
L2392: return { ok: false, error: "thread_open_hydration_timeout", scrolled: i, matched_selector: cardSelector };
L2393: }
L2394: const guard = await runWrongThreadGuard(page, t, {
L2395: forensicAccountLogin,
L2396: stage: "post_click_route_validation",
L2397: requireComposer: true
L2398: });
L2399: if (!guard.ok) return guard;
L2400: return { ok: true, scrolled: i, matched_selector: cardSelector, hydrated: true };
L2401: }
L2402: }
L2403: const delta = await scrollSidebarShort(page).catch(() => 0);
L2404: await humanPause("scroll", "sidebar_scroll");
L2405: }
L2406:
L2407: let hrefPreview = [];
L2408: try {
L2409: hrefPreview = await page.$$eval("a[href]", (els) =>
L2410: els.map((e) => String(e.getAttribute("href") || "").trim()).filter(Boolean).filter((h) => h.includes("/messages")).slice(0, 25)

```

```

L2411:);
L2412: } catch (_) {}
L2413:
L2414: return { ok: false, error: "thread_card_not_found", href_preview: hrefPreview };
L2415:}
```

```

PILAR 3 - Rotina de Raspagem DOM e Envio de Saudação (`virtusDelta.js`)

```

```txt
L742:function generateDeltaGreeting() {
L743: try {
L744: const cfg = readAtendimentoDeltaConfigSync() || {};
L745: const horario = resolveSaudacaoHorarioToken();
L746: const bloco1Raw = _pick(cfg.bloco1);
L747: const bloco1 = String(bloco1Raw || "").replace(/\[saudacao_horario\]/gi, horario).trim();
L748:
L749: const bloco2 = _pick(cfg.bloco2);
L750: const bloco3 = _pick(cfg.bloco3);
L751: const bloco4 = _pick(cfg.bloco4);
L752:
L753: const out = [bloco1, bloco2, bloco3, bloco4]
L754: .map((s) => String(s || "").trim())
L755: .filter(Boolean)
L756: .join("\n\n");
L757: return out;
L758: } catch {
L759: return [
L760: "Olá, [saudacao_horario]! Está disponível sim.".replace(/\[saudacao_horario\]/gi, resolveSaudacaoHorarioToken()),
L761: "Temos atendimento rápido e valores competitivos.",
L762: "Trabalhamos com fretes de pequeno, médio e grande porte.",
L763: "Me conta o que você precisa transportar para eu te ajudar agora.",
L764:].join("\n\n");
L765: }
L766:}

L1863:async function extractMarketplaceItemLink(page) {
L1864: const href = await page.evaluate(() => {
L1865: const host = location.origin || "";
L1866: const a =
L1867: document.querySelector('div[class*="x1a8lsjc"] a[href*="/marketplace/item/"]') ||
L1868: document.querySelector('a[href*="/marketplace/item/"]');
L1869: if (!a) return "";
L1870: const h = String(a.getAttribute('href') || "").trim();
L1871: if (!h) return "";
L1872: if (h.startsWith('http')) return h;
L1873: return host + h;
L1874: }).catch(() => "");
L1875: return String(href || "").trim();
L1876:}

L2891:async function collectCityFromItemLinkUsingGlobalCollector({ itemLink, threadKey, accountLogin }) {
L2892: if (typeof getDeltaCityCollector !== "function") {
L2893: return { ok: false, error: "delta_city_collector_unavailable" };

```

```

L2894: }
L2895: const collector = await getDeltaCityCollector();
L2896: if (!collector || typeof collector.collectCityFromItemLink !== "function") {
L2897: return { ok: false, error: "delta_city_collector_runtime_invalid" };
L2898: }
L2899: const out = await collector.collectCityFromItemLink({
L2900: item_link: itemLink,
L2901: thread_key: threadKey,
L2902: account_login: accountLogin,
L2903: });
L2904: return out && typeof out === "object" ? out : { ok: false, error: "delta_city_collector_unknown_error" };
L2905:}
L3919: async function sendDeltaGreetingNow({ threadKey, mensagensCliente }) {
L3920: if (!running || !epochOk()) return { ok: false, error: "delta_runtime_not_ready" };
L3921: const t = String(threadKey || "").trim();
L3922: if (!t) return { ok: false, error: "missing_thread_key" };
L3923:
L3924: const mensagensConcatenadas = String(mensagensCliente || "").replace(/\r/g, "").trim();
L3925: await enforceGlobalDeltaCooldown(ACCOUNT_LOGIN);
L3926:
L3927: const prior = greetingStateByThread.get(t) || null;
L3928: const greetingAlreadySent = !!(prior && prior.sentAt);
L3929: const greetingText = String((prior && prior.greetingText) || generateDeltaGreeting() || "").trim();
L3930:
L3931: let itemLinkResolved = false;
L3932: let itemLinkResolver = null;
L3933: const itemLinkPromise = new Promise((resolve) => { itemLinkResolver = resolve; });
L3934: const resolveItemLink = (link) => {
L3935: if (itemLinkResolved) return;
L3936: itemLinkResolved = true;
L3937: itemLinkResolver(String(link || "").trim() || null);
L3938: };
L3939:
L3940: const cityCollectionPromise = (async () => {
L3941: const preferredLink = String((prior && prior.itemLink) || "").trim() || null;
L3942: const itemLink = preferredLink || (await itemLinkPromise);
L3943: if (!itemLink) return { ok: false, error: "item_link_missing" };
L3944: return await collectCityFromItemLinkUsingGlobalCollector({
L3945: itemLink,
L3946: threadKey: t,
L3947: accountLogin: ACCOUNT_LOGIN,
L3948: });
L3949: }).catch((e) => ({
L3950: ok: false,
L3951: error: (e && e.message) ? String(e.message) : "city_collect_exception",
L3952: }));
L3953:
L3954: let sendOut = null;
L3955: if (!greetingAlreadySent) {
L3956: sendOut = await sendReplyFlow({
L3957: page,
L3958: threadKey: t,

```

```

L3959: textoResposta: greetingText,
L3960: fromNetworkLead: true,
L3961: onItemLink: (link) => resolveItemLink(link),
L3962: });
L3963: if (sendOut && sendOut.item_link) resolveItemLink(sendOut.item_link);
L3964: else resolveItemLink(null);
L3965: if (!sendOut || !sendOut.ok) {
L3966: return { ok: false, error: String((sendOut && sendOut.error) || "hands_send_failed") };
L3967: }
L3968: } else {
L3969: if (!(prior && prior.itemLink)) {
L3970: const openOut = await openThreadAndExtractItemLink(page, t, { fromNetworkLead: true });
L3971: if (openOut && openOut.ok && openOut.item_link) resolveItemLink(openOut.item_link);
L3972: else resolveItemLink(null);
L3973: } else {
L3974: resolveItemLink(prior.itemLink);
L3975: }
L3976: sendOut = { ok: true, item_link: (prior && prior.itemLink) || null };
L3977: }
L3978:
L3979: const cityCollectMaxWaitMs = Math.max(4_000, Number(process.env.VIRTUS_DELTA_CITY_COLLECT_MAX_WAIT));
L3980: const cityOut = await Promise.race([
L3981: cityCollectionPromise,
L3982: sleep(cityCollectMaxWaitMs).then(() => ({ ok: false, error: "city_collect_timeout", timeout_ms: cityCollectMaxWaitMs })),
L3983:]);
L3984:
L3985: if (!cityOut || cityOut.ok !== true || !String(cityOut.cidade || "").trim()) {
L3986: const itemLinkToKeep = String((sendOut && sendOut.item_link) || (prior && prior.itemLink) || "").trim() || null;
L3987: greetingStateByThread.set(t, {
L3988: sentAt: Number((prior && prior.sentAt) || Date.now()),
L3989: greetingText,
L3990: itemLink: itemLinkToKeep,
L3991: city: null,
L3992: citySource: null,
L3993: });
L3994: return {
L3995: ok: false,
L3996: error: String((cityOut && cityOut.error) || "city_collect_failed"),
L3997: greeting_already_sent: true,
L3998: greeting_text: greetingText,
L3999: };
L4000: }
L4001:
L4002: const cityCandidate = String(cityOut.cidade || "").trim();
L4003: const citySource = String(cityOut.city_source || "collector_listing_page").trim();
L4004: const itemLinkFinal = String((sendOut && sendOut.item_link) || (prior && prior.itemLink) || "").trim() || null;
L4005:
L4006: greetingStateByThread.set(t, {
L4007: sentAt: Number((prior && prior.sentAt) || Date.now()),
L4008: greetingText,
L4009: itemLink: itemLinkFinal,
L4010: city: cityCandidate,

```

```

L4011: citySource,
L4012: });
L4013:
L4014: let profileUrl = null;
L4015: try {
L4016: const u = String(page && page.url ? page.url() : "").trim();
L4017: if (u) profileUrl = u;
L4018: } catch (_) {}
L4019: let nomeClienteLimpo = null;
L4020: try { nomeClienteLimpo = await extractLeadClientNameFromFeedDom(page); } catch (_) {}
L4021:
L4022: const nowTs = Date.now();
L4023: writeLastDeltaSendTimestamp(ACCOUNT_LOGIN, nowTs);
L4024: lastCrossThreadKey = String(t);
L4025: lastCrossThreadSendAt = nowTs;
L4026:
L4027: return {
L4028: ok: true,
L4029: cidade: cityCandidate || null,
L4030: city_source: citySource,
L4031: profile_url: profileUrl,
L4032: greeting_text: greetingText,
L4033: mensagens_cliente: mensagensConcatenadas,
L4034: nome_cliente_limpo: nomeClienteLimpo,
L4035: customer_name: nomeClienteLimpo,
L4036: };
L4037: }
```

```

PILAR 4 - Acoplamento de Filas e Proteção Anti-Bloqueio (`ctrl.virtus`)

`worker.js` (`\_\_deltaResolveVirtusRunner`, `\_\_deltaRunHandsGreetingFlow`, dispatch reply IPC)

```

```txt
L16006:async function __deltaResolveVirtusRunner(nome, { need = 'greeting' } = {}) {
L16007: try {
L16008: const ctrl = controllers.get(nome);
L16009: if (!ctrl || !ctrl.browser || !ctrl.browser.isConnected?()) return null;
L16010: const needMode = String(need || "").trim().toLowerCase();
L16011: const isReplyNeed = needMode === 'reply';
L16012: const requiredFn = isReplyNeed ? 'enqueueDeltaReply' : 'enqueueDeltaGreetingFlow';
L16013: const resolveRunner = async (candidate) => {
L16014: try {
L16015: return (candidate && typeof candidate.then === 'function')
L16016: ? await candidate.catch(() => null)
L16017: : (candidate || null);
L16018: } catch {
L16019: return null;
L16020: }
L16021: };
L16022:
L16023: const direct = await resolveRunner(ctrl.virtus);

```

```

L16024: if (direct && typeof direct[requiredFn] === 'function') return direct;
L16025:
L16026: const networkEvidenceAt = Number(ctrl.deltaNetworkEvidenceAt || 0) || 0;
L16027: if (!isReplyNeed && !networkEvidenceAt) return null;
L16028:
L16029: const pageUrl = () => {
L16030: try { return String(ctrl.mainPage && ctrl.mainPage.url ? ctrl.mainPage.url() : "").trim().toLowerCase(); } catch { return
L16031: }}();
L16032: if (pageUrl && !/facebook\.com|messenger\.com/.test(pageUrl)) return null;
L16033:
L16034: if (!deltaVirtus || typeof deltaVirtus.startVirtusDeltaRuntime !== 'function') {
L16035: loadDeltaVirtusRuntime();
L16036: }
L16037: if (!deltaVirtus || typeof deltaVirtus.startVirtusDeltaRuntime !== 'function') return null;
L16038:
L16039: const bypassInterlockForReply = !isReplyNeed;
L16040: const bootInterlockEnabled = String(process.env.DELTA_BOOT_INTERLOCK_ENABLED || '1').trim() !== '0';
L16041: const bootInterlockHoldMs = Math.max(3000, Number(process.env.DELTA_BOOT_INTERLOCK_HOLD_MS || 3000));
L16042:
L16043: ctrl.virtus = deltaVirtus.startVirtusDeltaRuntime(ctrl.browser, nome, {
L16044: epoch: ctrl.virtusEpoch || 0,
L16045: slowMode: false,
L16046: governorMode: 'full',
L16047: restrictTab: 0,
L16048: bootReason: 'delta_unified_runtime',
L16049: bootInterlockEnabled: bypassInterlockForReply ? false : bootInterlockEnabled,
L16050: bootInterlockHoldMs: bypassInterlockForReply ? 0 : bootInterlockHoldMs,
L16051: bootInterlockBeforeNavigate: bypassInterlockForReply ? null : ({ page }) => __deltaPrepareBootInterlockEar(nome,
L16052: bootInterlockIsEarReady: bypassInterlockForReply ? null : () => __deltalsBootEarReady(nome),
L16053: });
L16054: const booted = await resolveRunner(ctrl.virtus);
L16055: if (booted && typeof booted[requiredFn] === 'function') return booted;
L16056: return null;
L16057: } catch {
L16058: return null;
L16059: }
L16060:}
L16084:async function __deltaRunHandsGreetingFlow({ nome, threadKey, mensagensCliente }) {
L16085: const runner = await __deltaResolveVirtusRunner(nome, { need: 'greeting' });
L16086: if (!runner || typeof runner.enqueueDeltaGreetingFlow !== 'function') {
L16087: return { ok: false, error: 'delta_hands_unavailable' };
L16088: }
L16089: try {
L16090: return await runner.enqueueDeltaGreetingFlow({
L16091: thread_key: String(threadKey || "").trim(),
L16092: mensagens_cliente: String(mensagensCliente || "")
L16093: });
L16094: } catch (e) {
L16095: return { ok: false, error: (e && e.message) ? e.message : String(e) };
L16096: }
L16097:}
L10528: const ctrl = controllers.get(n);

```

```

L10529: if (!ctrl || !ctrl.browser || !ctrl.browser.isConnected?.()) {
L10530: if (cmid) __deltaReplyIngressRelease(n, cmid);
L10531: return { ok: false, error: 'browser_not_connected' };
L10532: }
L10533: const runner = await __deltaResolveVirtusRunner(n, { need: 'reply' });
L10534: if (!runner || typeof runner.enqueueDeltaReply !== 'function') {
L10535: if (cmid) __deltaReplyIngressRelease(n, cmid);
L10536: return { ok: false, error: 'delta_hands_unavailable' };
L10537: }
L10538: try {
L10539: Promise.resolve()
L10540: .then(() => runner.enqueueDeltaReply({ thread_key: tk, texto_resposta: tr, client_message_id: cmid }))
L10541: .then((out) => {
L10542: if (cmid) __deltaReplyIngressRelease(n, cmid);
L10543: })
L10544: .catch((e) => {
L10545: if (cmid) __deltaReplyIngressRelease(n, cmid);
L10546: });
L10547: } catch {
L10548: if (cmid) __deltaReplyIngressRelease(n, cmid);
L10549: }
L10550: return { ok: true, status: 'queued', client_message_id: cmid };
 ``

```

#### `virtusDelta.js` (`createSerialQueue`, runtime queue única, `enqueueDeltaReply`, `enqueueDeltaGreetingFlow`)

```

 ``txt
L2907: function createSerialQueue() {
L2908: let chain = Promise.resolve();
L2909: let depth = 0;
L2910: let maxDepth = 0;
L2911: let lastEnqueueAt = 0;
L2912: let lastDequeueAt = 0;
L2913: let lastDoneAt = 0;
L2914:
L2915: const enqueue = (fn) => {
L2916: const enqueuedAt = Date.now();
L2917: depth = Math.max(0, depth + 1);
L2918: maxDepth = Math.max(maxDepth, depth);
L2919: lastEnqueueAt = enqueuedAt;
L2920: chain = chain
L2921: .then(async () => {
L2922: lastDequeueAt = Date.now();
L2923: try {
L2924: return await fn();
L2925: } finally {
L2926: depth = Math.max(0, depth - 1);
L2927: lastDoneAt = Date.now();
L2928: }
L2929: })
L2930: .catch(() => {});
L2931: return chain;

```

```

L2932: };
L2933: enqueue.getDepth = () => depth;
L2934: enqueue.getMaxDepth = () => maxDepth;
L2935: enqueue.getMeta = () => ({ depth, maxDepth, lastEnqueueAt, lastDequeueAt, lastDoneAt });
L2936: return enqueue;
L2937:}
L3115: const enqueue = createSerialQueue();
L3116: const autoGreetingEnabled = String(process.env.VIRTUS_DELTA_AUTO_GREETING || "1").trim() === "1";
L3117: const autoGreetingSentThreads = new Set(); // threadKey
L3118: const autoGreetingTimers = new Map(); // threadKey -> Timeout
L3119: const greetingStateByThread = new Map(); // threadKey -> { sentAt, greetingText, itemLink, city, citySource }
L4106: const enqueueDeltaReply = ({ thread_key, texto_resposta, client_message_id, _requeue_count = 0 } = {}) => {
L4107: return enqueue(async () => {
L4108: try {
L4109: const tk = String(thread_key || "").trim();
L4110: const tr = String(texto_resposta || "").replace(/r/g, "");
L4111: const cmid = String(client_message_id || "").trim() || null;
L4112: const out = await sendDeltaReplyNow({ threadKey: tk, textoResposta: tr, clientId: cmid });
L4113: let requeueScheduled = false;
L4114: try {
L4115: const ok = !(out && out.error);
L4116: const err = String(out && out.error || "").trim();
L4117: const isSelectorLike =
L4118: err === "composer_missing" ||
L4119: err === "thread_card_not_found" ||
L4120: err === "thread_open_hydration_timeout" ||
L4121: err === "send_not_confirmed_composer_not_empty" ||
L4122: err === "composer_text_not_registered";
L4123: const tries = Math.max(0, Number(_requeue_count || 0) || 0);
L4124: if (!ok && isSelectorLike && tries < 2) {
L4125: const nextCount = tries + 1;
L4126: const delayMs = randomBetween(1200, 2400);
L4127: try {
L4128: setTimeout(() => {
L4129: enqueueDeltaReply({ thread_key: tk, texto_resposta: tr, client_message_id: cmid, _requeue_count: nextCount
L4130: .catch(() => {});
L4131: }, delayMs).unref?.();
L4132: requeueScheduled = true;
L4133: } catch (_) {}
L4134: }
L4135: } catch (_) {}
L4136: if (requeueScheduled) return out;
L4137: return out;
L4138: } catch (e) {
L4139: return { ok: false, error: e && e.message ? e.message : String(e) };
L4140: }
L4141: });
L4142: };
L4183: const enqueueDeltaGreetingFlow = ({ thread_key, mensagens_cliente }) => {
L4184: return enqueue(async () => {
L4185: try {
L4186: return await sendDeltaGreetingNow({

```



```

L4187: threadKey: thread_key,
L4188: mensagensCliente: mensagens_cliente,
L4189: });
L4190: } catch (e) {
L4191: return { ok: false, error: e && e.message ? e.message : String(e) };
L4192: }
L4193: });
L4194: };
...

```

## BLOCO CRU POS-REPARO - Instrumentacao e Interlock

### `worker.js` - barramento isolado + enqueue/pump sequencial

```

``txt
L16:const FORENSIC_TRIAGEM_LOG_PATH = path.join(__dirname, '..', 'dados', 'forensic_triagem.log');
L76:const FORENSIC_TRIAGEM_ROTATE_MAX_BYTES = 10 * 1024 * 1024; // 10MB hard ceiling (circular)
L77:function __triagemCircularAppendSync(signature, details = null) {
L78: try {
L79: const sig = String(signature || "").trim();
L80: if (!sig) return false;
L81: const fp = String(FORENSIC_TRIAGEM_LOG_PATH || "").trim();
L82: if (!fp) return false;
L83: const payload = (details && typeof details === 'object')
L84: ? { ...details }
L85: : { message: String(details || "") };
L86: const line = `[$${sig}] ${JSON.stringify({ timestamp: Date.now(), ...payload })}\n`;
L87: const lineBytes = Buffer.byteLength(line, 'utf8');
L88: try { fs.mkdirSync(path.dirname(fp), { recursive: true }); } catch {}
L89:
L90: let currentSize = 0;
L91: try {
L92: if (fs.existsSync(fp)) {
L93: const st = fs.statSync(fp);
L94: currentSize = Number(st && st.size || 0) || 0;
L95: }
L96: } catch {}
L97:
L98: if ((currentSize + lineBytes) > FORENSIC_TRIAGEM_ROTATE_MAX_BYTES) {
L99: const keepBytes = Math.max(0, FORENSIC_TRIAGEM_ROTATE_MAX_BYTES - lineBytes);
L100: let tail = "";
L101: if (keepBytes > 0 && currentSize > 0) {
L102: let fd = null;
L103: try {
L104: fd = fs.openSync(fp, 'r');
L105: const start = Math.max(0, currentSize - keepBytes);
L106: const toRead = Math.max(0, currentSize - start);
L107: if (toRead > 0) {
L108: const buf = Buffer.allocUnsafe(toRead);
L109: const got = fs.readSync(fd, buf, 0, toRead, start);
L110: tail = buf.slice(0, Math.max(0, got)).toString('utf8');
L111: }

```

```

L112: } catch {
L113: tail = "";
L114: } finally {
L115: try { if (fd) fs.closeSync(fd); } catch {}
L116: }
L117: }
L118: fs.writeFileSync(fp, tail + line, 'utf8');
L119: return true;
L120: }
L121:
L122: fs.appendFileSync(fp, line, 'utf8');
L123: return true;
L124: } catch {
L125: return false;
L126: }
L127:}
L128:function __deltaLogTriagemWorker(details = null) {
L129: return __triagemCircularAppendSync('FORENSIC_TRIAGEM_WORKER', details);
L130:}
L15108:function __deltaEnqueueNewLeadTimerToDiskSync({ nome, thread_key, dueAt, delayMs, forceEnqueue = false, queueLane }) {
L15109: try {
L15110: const n = String(nome || "").trim();
L15111: const tk = String(thread_key || "").trim();
L15112: if (!n || !tk) return false;
L15113: if (!forceEnqueue && !__deltaCanEnqueueNewLeadTimerFromDiskSync(n, tk)) return false;
L15114: if (__deltaHasPendingNewLeadTimerOnDiskSync(n, tk)) return false;
L15115: __deltaEnsureNewLeadsQueueDirsSync(n);
L15116: const rec = {
L15117: ts: Date.now(),
L15118: type: 'new_lead_timer',
L15119: nome: n,
L15120: thread_key: tk,
L15121: delayMs: Math.max(0, Number(delayMs || 0) || 0),
L15122: dueAt: Math.max(0, Number(dueAt || 0) || 0),
L15123: queue_lane: String(queueLane || 'primary').trim() || 'primary',
L15124: retry_reason: String(retryReason || "").trim() || null,
L15125: };
L15126: fs.appendFileSync(__deltaNewLeadsOutboxPath(n), JSON.stringify(rec) + '\n', 'utf8');
L15127: try {
L15128: __deltaLogTriagemWorker({
L15129: event: 'timer_enqueued_in_reservoir',
L15130: account_login: n,
L15131: thread_key: tk,
L15132: due_at: Number(rec.dueAt || 0) || 0,
L15133: queue_lane: String(rec.queue_lane || ""),
L15134: });
L15135: } catch {}
L15136: return true;
L15137: } catch {
L15138: return false;
L15139: }
L15140:}

```

```

L15188:async function __deltaRunNewLeadsTimerPump(nome) {
L15189: const n = String(nome || "").trim();
L15190: if (!n) return;
L15191: if (__deltaNewLeadsTimerInFlight.has(n)) return; // 1 timer por vez POR CONTA
L15192: try {
L15193: const outbox = __deltaNewLeadsOutboxPath(n);
L15194: if (!fs.existsSync(outbox)) return;
L15195: const cursor = __deltaReadNewLeadsCursorSync(n);
L15196: const offset = Math.max(0, Number(cursor && cursor.offset || 0) || 0);
L15254: const handle = setTimeout(() => {
L15255: __deltaNewLeadsTimerInFlight.delete(n);
L15256: try {
L15257: __deltaLogTriagemWorker({
L15258: event: 'timer_expired_dispatching_downstream',
L15259: account_login: n,
L15260: thread_key: tk,
L15261: due_at: dueAt || null,
L15262: });
L15263: } catch {}
L15282: // Downstream: dispara a ação no pipeline soberano (ctrl.virtus) em background.
L15283: try {
L15284: Promise.resolve()
L15285: .then(() => __deltaHandleBufferedThreadTimer(n, tk, { reason: 'initial' })))
L15286: .catch(() => {})
L15287: .finally(() => {
L15288: try { __deltaKickNewLeadsTimerPump(n); } catch {}
L15289: });
L15290: } catch {}
L15291: try { __deltaKickNewLeadsTimerPump(n); } catch {}
L15292: }
L15293: }, Math.max(0, remainMs));
 }
}

```

### `virtusDelta.js` - barramento DOM + corte de fallback por teclado em `openThreadByClick`

```

``txt
L11:const FORENSIC_TRIAGEM_LOG_PATH = path.join(__dirname, "..", "dados", "forensic_triagem.log");
L75:const FORENSIC_TRIAGEM_ROTATE_MAX_BYTES = 10 * 1024 * 1024; // 10MB hard ceiling (circular)
L76:function __triagemCircularAppendSync(signature, details = null) {
L77: try {
L78: const sig = String(signature || "").trim();
L79: if (!sig) return false;
L80: const fp = String(FORENSIC_TRIAGEM_LOG_PATH || "").trim();
L81: if (!fp) return false;
L82: const payload = (details && typeof details === "object")
L83: ? { ...details }
L84: : { message: String(details || "") };
L85: const line = `[${sig}] ${JSON.stringify({ timestamp: Date.now(), ...payload })}\n`;
L86: const lineBytes = Buffer.byteLength(line, "utf8");
L87: try { fsSync.mkdirSync(path.dirname(fp), { recursive: true }); } catch (_) {}
L88: let currentSize = 0;
L89: try {

```

```

L90: if (fsSync.existsSync(fp)) {
L91: const st = fsSync.statSync(fp);
L92: currentSize = Number(st && st.size || 0) || 0;
L93: }
L94: } catch () {}
L95: if ((currentSize + lineBytes) > FORENSIC_TRIAGEM_ROTATE_MAX_BYTES) {
L96: const keepBytes = Math.max(0, FORENSIC_TRIAGEM_ROTATE_MAX_BYTES - lineBytes);
L97: let tail = "";
L98: if (keepBytes > 0 && currentSize > 0) {
L99: let fd = null;
L100: try {
L101: fd = fsSync.openSync(fp, "r");
L102: const start = Math.max(0, currentSize - keepBytes);
L103: const toRead = Math.max(0, currentSize - start);
L104: if (toRead > 0) {
L105: const buf = Buffer.allocUnsafe(toRead);
L106: const got = fsSync.readSync(fd, buf, 0, toRead, start);
L107: tail = buf.slice(0, Math.max(0, got)).toString("utf8");
L108: }
L109: } catch () {
L110: tail = "";
L111: } finally {
L112: try { if (fd) fsSync.closeSync(fd); } catch () {}
L113: }
L114: }
L115: fsSync.writeFileSync(fp, tail + line, "utf8");
L116: return true;
L117: }
L118: fsSync.appendFileSync(fp, line, "utf8");
L119: return true;
L120: } catch () {
L121: return false;
L122: }
L123:}

L127:function __deltaLogTriagemDom(details = null) {
L128: return __triagemCircularAppendSync("FORENSIC_TRIAGEM_DOM", details);
L129:}

L2332:async function openThreadByClick(page, threadKey, { maxScrollSteps = 16, forensicAccountLogin = null } = {}) {
L2333: const t = String(threadKey || "").trim();
L2334: if (!t) throw new Error("thread_key_empty");
L2335: try {
L2336: const urlInicial = String(page && page.url ? page.url() : "").trim() || null;
L2337: __deltaLogTriagemDom({
L2338: stage: "automation_start",
L2339: thread_key: t,
L2340: url_inicial: urlInicial,
L2341: });
L2342: } catch () {}
L2470: const points = (clickPlan && clickPlan.ok && Array.isArray(clickPlan.points)) ? clickPlan.points : [];
L2471: for (const p of points.slice(0, 2)) {
L2472: const px = Number(p && p.x);
L2473: const py = Number(p && p.y);

```

```

L2474: if (!Number.isFinite(px) || !Number.isFinite(py)) continue;
L2475: try {
L2476: await page.mouse.move(px, py, { steps: 6 });
L2477: await page.mouse.click(px, py, { delay: 100 });
L2478: } catch () {
L2479: continue;
L2480: }
L2496: try {
L2497: await page.waitForFunction(
L2498: (threadId) => {
L2499: const path = String(location.pathname || "");
L2500: return path.includes("/messages") && path.includes(`/t/${threadId}`);
L2501: },
L2502: { timeout: 1800 },
L2503: t
L2504:);
L2505: openedByNavigation = true;
L2506: break;
L2507: } catch () {}
L2508: }
L2509: if (!openedByNavigation) {
L2510: try {
L2511: __deltaLogTriagemDom({
L2512: stage: "bounding_click_failed_abort",
L2513: thread_key: t,
L2514: selector: cardSelector,
L2515: scrolled: i,
L2516: retry_queue_hint: "async_disk_retry",
L2517: });
L2518: } catch () {}
L2519: return {
L2520: ok: false,
L2521: error: "thread_card_not_found",
L2522: scrolled: i,
L2523: matched_selector: cardSelector,
L2524: retry_queue_hint: "async_disk_retry"
L2525: };
L2526: }
L2529: // Regra rígida: após clicar no card (chat fechado), aguardar hidratação do composer Lexical.
L2530: await page.waitForSelector('div[data-lexical-editor="true"]', { timeout: 5000 });
L2531: try {
L2532: const urlFinal = String(page && page.url ? page.url() : "").trim() || null;
L2533: __deltaLogTriagemDom({
L2534: stage: "composer_hydration_success",
L2535: thread_key: t,
L2536: selector: cardSelector,
L2537: scrolled: i,
L2538: url_final: urlFinal,
L2539: });
L2540: } catch () {}

```